

## Dashboarding: Splunk vs Python

Splunk è potentissimo come motore di ricerca e indicizzazione, ma le sue dashboard native hanno limiti precisi, e Python è proprio lo strumento giusto per superarli.

### Cosa Splunk monitora nativamente, e in che modo

Splunk lavora su **eventi indicizzati con un timestamp**: qualunque cosa abbia un tempo e del testo (log, metriche, trap SNMP, eventi syslog) viene ingerita, indicizzata e resa interrogabile con il linguaggio SPL. Su questa base calcola nativamente diverse famiglie di KPI.

I **KPI di volume e frequenza** sono il pane quotidiano: conteggi di eventi per unità di tempo (richieste al minuto, errori all'ora), tassi e rapporti (error rate come errori/totale), distribuzioni per categoria (stats count by host status). Qui Splunk è eccellente, perché è esattamente ciò per cui è nato.

I **KPI di latenza e prestazioni** li calcola se i tempi sono nei dati: medie, percentili (perc95, perc99), min/max, deviazione standard, con i comandi stats e timechart. Funzionano bene su finestre temporali, e il timechart produce serie storiche pronte da graficare.

I **KPI di disponibilità e affidabilità** sono ricavabili — uptime/downtime contando heartbeat mancanti, conteggio di errori critici, SLO espressi come percentuale di richieste sotto soglia. Splunk ha anche un modulo dedicato, **ITSI** (IT Service Intelligence), che introduce il concetto di *KPI* e *glass table* come oggetti di prima classe, con soglie e severità — ma è un prodotto a parte, costoso, non incluso nel Splunk base.

Per il **rilevamento di anomalie** Splunk offre comandi come anomalydetection, trendline, e il **Machine Learning Toolkit (MLTK)**, che porta dentro algoritmi (clustering, forecasting, outlier detection). Esiste, ma è un'estensione, con una sua curva di apprendimento.

Il modo in cui tutto questo diventa visibile sono le **dashboard native** (Simple XML o le più recenti Dashboard Studio): pannelli che incapsulano una ricerca SPL e la rendono come tabella, grafico a linee, a barre, singola statistica, mappa.

### Dove arrivano i limiti (e perché può servire Python)

I limiti delle dashboard native non sono nel *calcolo* — l'SPL calcola moltissimo — ma nella **flessibilità di visualizzazione, nell'integrazione tra sorgenti e nell'analisi statistica avanzata**. Vediamoli, perché è qui che la tua domanda diventa interessante.

Il limite più sentito è la **libertà visuale**: le dashboard Splunk offrono un insieme fisso di pannelli. Appena serve un tipo di grafico che non c'è — una heatmap di correlazione tra KPI, un correlogramma (l'ACF di cui parlavamo), un grafico a violino, una *small multiples* di decine di host affiancati, un grafico con annotazioni di incidenti sovrapposte — si è bloccati, o si deve ricorrere a estensioni JavaScript custom dentro Splunk, scomode da mantenere.

Il secondo limite è l'**analisi statistica sofisticata**. L'SPL fa percentili e medie mobili benissimo, ma correlazione di Pearson/Spearman tra molte metriche, cross-correlazione con lag (il "quale KPI anticipa quale" che abbiamo visto), regressioni, scomposizione stagionale, intervalli di confidenza, test statistici — sono goffi o impossibili in SPL puro, mentre sono una riga in pandas/scipy.

Il terzo è l'**integrazione con sorgenti esterne a Splunk**. Una dashboard nativa vede solo ciò che è in Splunk. Ma un KPI di business spesso nasce *incrociando* i dati Splunk con un database (Oracle/SQL Server), un

CMDB, un foglio di costi, l'output di un'API. Combinare "errori da Splunk" con "ricavi persi da un DB" in un unico indicatore è terreno naturale di Python, non di una dashboard Splunk.

Il quarto è la **logica condizionale complessa e il calcolo di KPI compositi**: indici che pesano e normalizzano più metriche (un "health score" che combina latenza, errori e saturazione con pesi diversi per servizio), con regole che in SPL diventano una catena di eval illeggibile, mentre in Python sono funzioni chiare e testabili — esattamente la filosofia del corso.

Infine ci sono limiti **pratici**: la personalizzazione fine dell'aspetto (branding, layout per la dirigenza), l'esportazione in formati specifici (un PDF mensile per il management, un Excel per i revisori), e il fatto che le dashboard avanzate o ITSI hanno un **costo di licenza** che non tutti hanno.

## Come Python può migliorarle e integrarle

Python può migliorare molto le dashboard di Splunk. Il principio architetturale giusto è una **divisione del lavoro**: lasciare a Splunk ciò che fa meglio — ingestione, indicizzazione, ricerca su grandi volumi — e usare Python per ciò che a Splunk manca. Concretamente si delineano alcune strategie complementari.

La più diretta è **Splunk come fonte, Python come cervello e vetrina**: Python estrae i risultati di una ricerca SPL via REST (lo abbiamo visto nel modulo 3.2 — la ricerca restituisce JSON), li carica in pandas, vi applica l'analisi statistica avanzata che SPL non offre, e li visualizza in una dashboard costruita con strumenti Python. Splunk fa il *heavy lifting* dell'aggregazione su miliardi di eventi (efficiente, vicino ai dati), Python riceve un risultato già ridotto e ci lavora con piena libertà. Questo è importante: non si scaricano gli eventi grezzi: si lascia che Splunk aggregi e si tira giù solo il risultato.

La seconda strategia è l'**arricchimento e re-immissione**: Python calcola un KPI composito o un punteggio di anomalia che Splunk da solo non saprebbe produrre, e lo **rispedisce a Splunk via HEC** (di nuovo, modulo 3.2) come nuova metrica. Così il KPI avanzato diventa a sua volta indicizzato e graficabile dentro Splunk, e si ottiene il meglio dei due mondi senza abbandonare l'ecosistema esistente.

Sul fronte della **vetrina** (le dashboard vere e proprie), l'ecosistema Python offre un ventaglio di opzioni con compromessi diversi, che vale la pena conoscere a livello di scelta architetturale: framework per app analitiche interattive con pochissimo codice (Streamlit, Dash, Gradio), librerie di grafici interattivi da incorporare ovunque (Plotly, Bokeh, Altair), notebook che diventano report o cruscotti (Jupyter con Voilà), e generazione di report statici per la dirigenza (PDF/Excel schedulati). La scelta dipende da chi è il pubblico: un cruscotto interattivo per il team operativo è una cosa, un PDF mensile per il management è un'altra.

Due avvertenze oneste, da mettere sul tavolo in aula. Primo: **non si reinventa Splunk**. Se un KPI è ben servito da una dashboard nativa, costruirlo in Python è lavoro sprecato e un secondo sistema da mantenere — Python ha senso solo dove aggiunge ciò che a Splunk manca davvero (analisi avanzata, integrazione cross-sorgente, visualizzazioni custom, report). Secondo: si introduce un **componente da gestire** — un servizio Python che gira, si autentica a Splunk, va monitorato a sua volta; non è gratis in termini operativi.

In sintesi: Splunk è fortissimo nel calcolare KPI di volume, latenza e disponibilità su grandi volumi e nel mostrarli in dashboard standard; i suoi limiti stanno nella visualizzazione libera, nella statistica avanzata, nell'integrazione con dati esterni e nei KPI compositi. Python non lo sostituisce ma lo **estende** secondo due flussi — *pull* per analizzare e visualizzare fuori, *push* (HEC) per reimmettere KPI avanzati dentro — ed è proprio la naturale continuazione del modulo 3.2 del corso. Se vuoi, posso trasformare questa analisi in una tabella di confronto "KPI / fattibile in Splunk nativo / valore aggiunto di Python", utile come slide riassuntiva.

| Famiglia di KPI                                             | Splunk nativo                                                                | Valore aggiunto di Python                                           |
|-------------------------------------------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Volume e frequenza (conteggi, tassi, error rate)            | <b>Forte</b> — stats, timechart                                              | Poco; semmai l'unione con sorgenti esterne                          |
| Latenza / percentili (p95, p99)                             | <b>Forte</b> — perc95, timechart                                             | Confronto tra molte serie, distribuzioni, annotazioni d'incidente   |
| Disponibilità / SLO / error budget                          | <b>Parziale</b> — ricavabile; KPI di prima classe solo in ITSI (a pagamento) | SLO compositi, <i>burn-rate</i> , esito sintetico per il management |
| Correlazione tra KPI (Pearson/Spearman, heatmap)            | <b>Debole</b> — goffo in SPL puro                                            | Nativo: pandas.corr, heatmap leggibile                              |
| Correlazione temporale con <i>lag</i> (cross-correlazione)  | <b>Assente</b> in SPL puro                                                   | Nativo: shift/ACF, "quale KPI anticipa quale"                       |
| Anomaly detection / forecasting                             | <b>Parziale</b> — solo con MLTK (estensione)                                 | scikit-learn / statsmodels, pieno controllo e trasparenza           |
| KPI compositi / <i>health score</i> (pesi, normalizzazione) | <b>Debole</b> — catene di eval illeggibili                                   | Funzioni chiare, testabili e versionabili                           |
| Integrazione cross-sorgente (DB, CMDB, API, costi)          | <b>Assente</b> — vede solo ciò che è indicizzato                             | Unisce Splunk + Oracle/SQL Server + API in un unico KPI             |
| Visualizzazione custom & report                             | <b>Parziale</b> — pannelli fissi, export limitato                            | Streamlit/Dash/Plotly; PDF/Excel schedulati per la dirigenza        |

Il messaggio della slide, in una riga: dove la colonna Splunk dice *Forte* (volume, latenza), Python aggiunge poco e non vale la pena duplicare; dove dice *Debole* o *Assente* (correlazioni, KPI compositi, integrazione cross-sorgente) è esattamente lì che Python ripaga — secondo i due flussi del modulo 3.2, *pull* per analizzare/visualizzare fuori e *push* (HEC) per reimmettere i KPI avanzati dentro Splunk.